

ORB = the Smogon calc, forked + auto-filled (the right way)

TL;DR: The Smogon damage calculator is MIT-licensed open source, and the official Champions calc is built from it. So instead of reimplementing it (which you rightly hate), we **fork it, self-host it, and add an auto-fill layer**. Because our copy is served from our own origin, the browser's cross-site block disappears — we *can* auto-populate it, unlike the hosted official one. Result: pixel-identical to Smogon's, functions identically, and fills itself from your live battle.

License (you're clear)

- Repo: <https://github.com/smogon/damage-calc>
- the Smogon damage calculator (the math engine) and the UI markup/logic are **MIT** — free to use, modify, self-host, even commercially. Keep the LICENSE file + attribution. That's the only obligation.

The build (runs on your machine — I can't build webpack in the sandbox)

```
# 1. get it
git clone https://github.com/smogon/damage-calc
cd damage-calc

# 2. the engine (calculation) lives in calc/ ; the site UI in src/
cd calc && npm install && npm run compile && cd ..      # builds the Smogon damage calculator
npm install && npm run build                          # builds the web UI

# 3. Champions: the official site serves champions.html?mode=champions.
#   Confirm your clone includes the champions mode data (it drives the SP stat
#   system + banlist). If the public repo lags the live champions build, the
#   champions data can be added as a gen/format mod — see src/js/ and the
#   "mode" handling; ABRA already has the Champions SP rules in
#   CHOMP/engine/champ-model.js to cross-check against.
```

The auto-fill layer (the only thing we add)

Two clean options, both same-origin so they Just Work on our hosted copy:

1. **Userscript** → **calc via** `postMessage / localStorage`. The CHOMP userscript already reads the live battle (mons, sets, HP, field). On our self-hosted calc page, add a tiny script that, on load and on each turn, reads that state (shared `localStorage` key `chomp-orb-state`, same origin) and calls the calc's existing setters to populate attacker/defender/field. No fork of the UI internals needed — just drive its public form state.

1. **URL/state import.** The calc already has an Import box; our page can pre-fill it from the battle and trigger import. Less elegant but trivial.

Sketch of the hook (drop into the forked site, same origin as the calc):

```
// orb-autofill.js – reads the battle state CHOMP wrote, fills the Smogon calc
function fillFromBattle(){
  const s = JSON.parse(localStorage.getItem('chomp-orb-state') || 'null');
  if (!s) return; // no live battle yet
  // s = { yourActive:{species,item,ability,moves,boosts,hp}, theirActive:{...}, weather, terrain, ...
  setPokemon('attacker', s.yourActive); // call the calc's own set functions
  setPokemon('defender', s.theirActive);
  setField(s.weather, s.terrain);
  calculate(); // the calc's existing recompute
}
window.addEventListener('storage', e => { if (e.key==='chomp-orb-state') fillFromBattle(); });
fillFromBattle();
```

Why this beats both current states

- **vs. our hand-built orb.html:** it is the Smogon calc — same look, same trusted math, zero "it doesn't look right."
- **vs. linking the official calc:** self-hosting makes it same-origin, so auto-fill is allowed. The official hosted one can never be auto-filled from outside (browser security), which is the wall we kept hitting.

Where this lands in the product

- **CHOMP userscript:** ORB is a minimizable dock that opens our forked calc (auto-filled) beside the battle.
- **ABRA site:** the Chomp room links to our hosted forked calc (and "copy my team" for the official one as a fallback).

Once you've cloned + built it and picked a host path, I can write the `orb-autofill.js` against the real form IDs and wire it into the CHOMP dock.